

Iterators

New For Loop

Lab 05

What is a
What is a
reference?



References

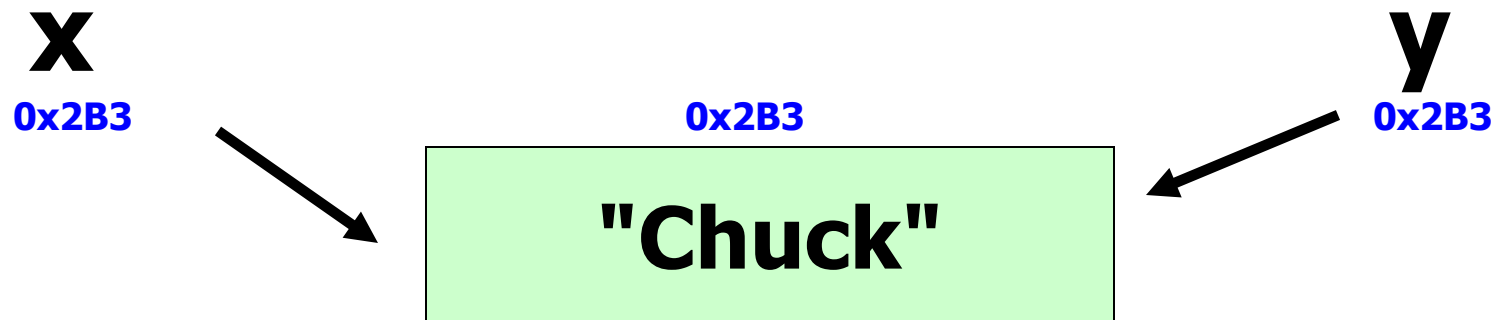
In Java, any variable that refers to an Object is a reference variable.

The variable stores the memory address of the actual Object.

References

```
String x = "Chuck";  
String y = x;
```

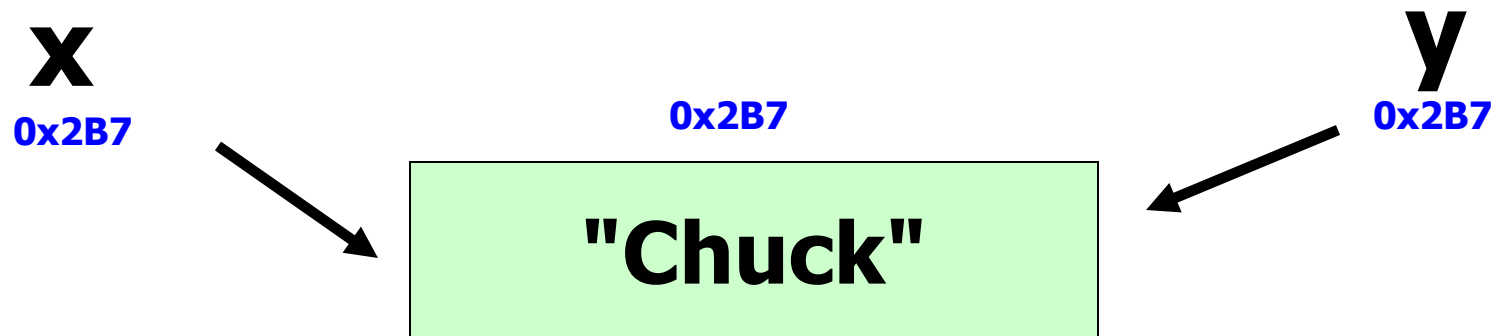
x and y store the same memory address.



References

```
String x = "Chuck";  
String y = "Chuck";
```

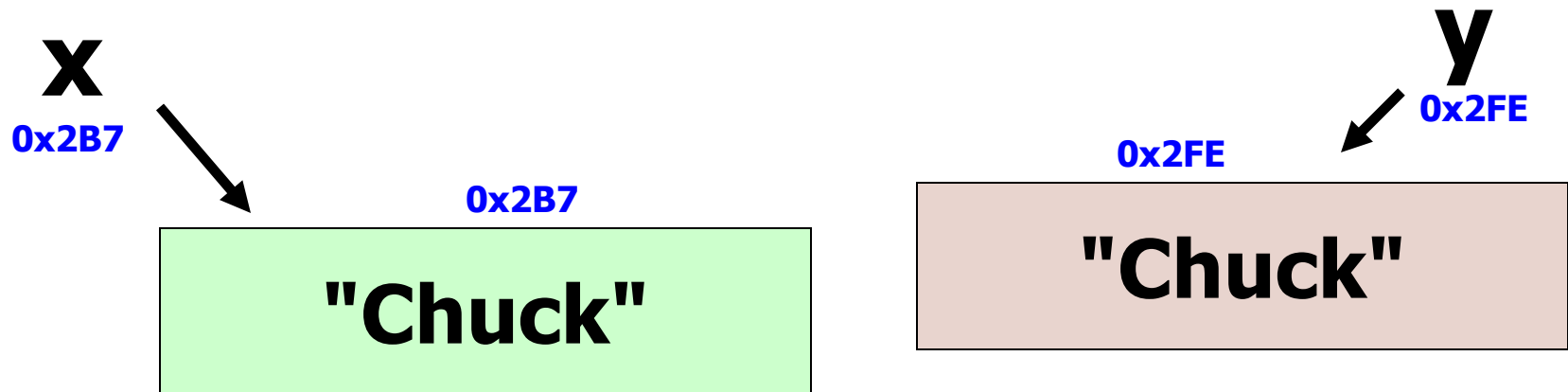
x and y store the same memory address.



References

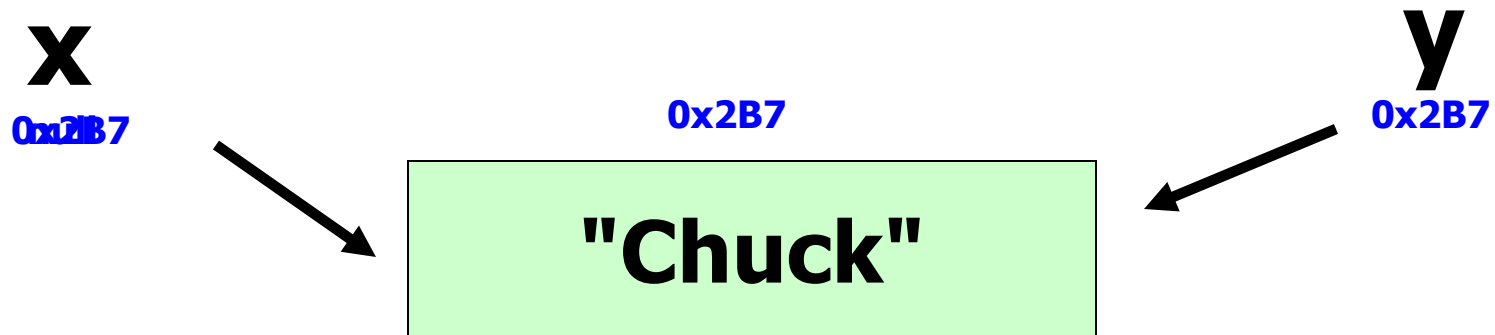
```
String x = new String("Chuck");  
String y = new String("Chuck");
```

x and y store different memory addresses.



References

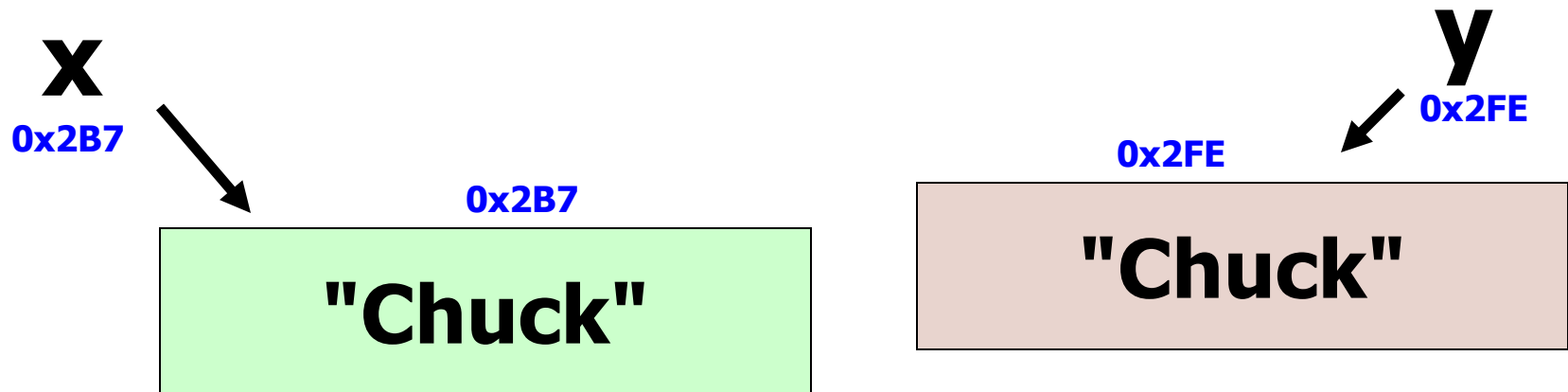
```
String x = "Chuck";  
String y = "Chuck";  
x = null;
```



References

```
String x = "Chuck";  
String y = new String("Chuck");
```

x and y store different memory addresses.





Contest Puzzlers

How many String object does this code create?

```
String x = new String("Chuck");  
String y = x;
```

Contest Puzzlers

```
String x = new String("Chuck");  
String y = x;
```

0x5E9A06

"Chuck"

x

0x2B3

0x2B3

"Chuck"

y

0x2B3



Contest Puzzlers

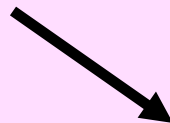
How many String object does this code create?

```
String x = new String("Chuck");  
String y = "Chuck";
```

Contest Puzzlers

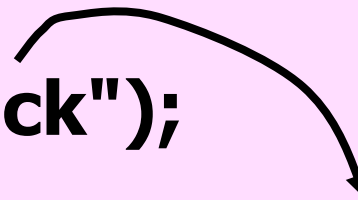
```
String x = new String("Chuck");  
String y = "Chuck";
```

x
0x2B3



"Chuck"

0x2B3



0x5E9A06

"Chuck"



y
0x2B3



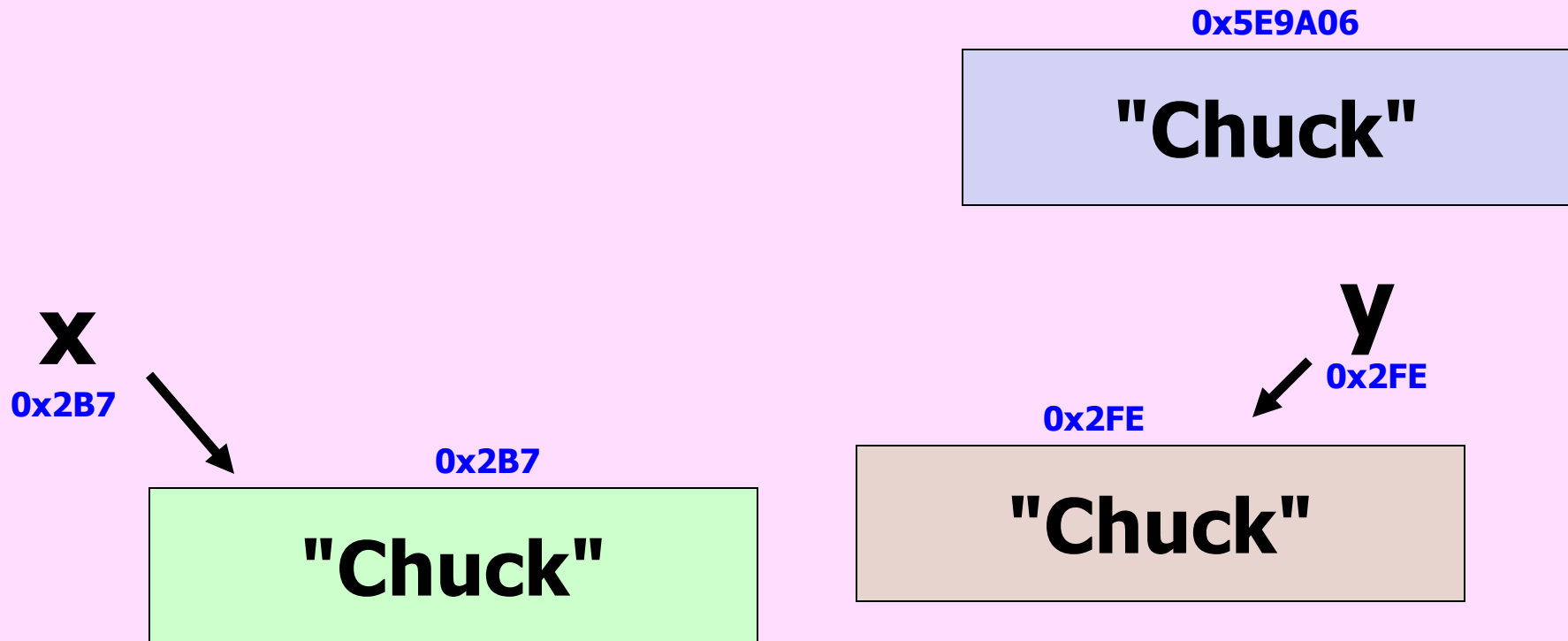
Contest Puzzlers

How many String object does this code create?

```
String x = new String("Chuck");  
String y = new String("Chuck");
```

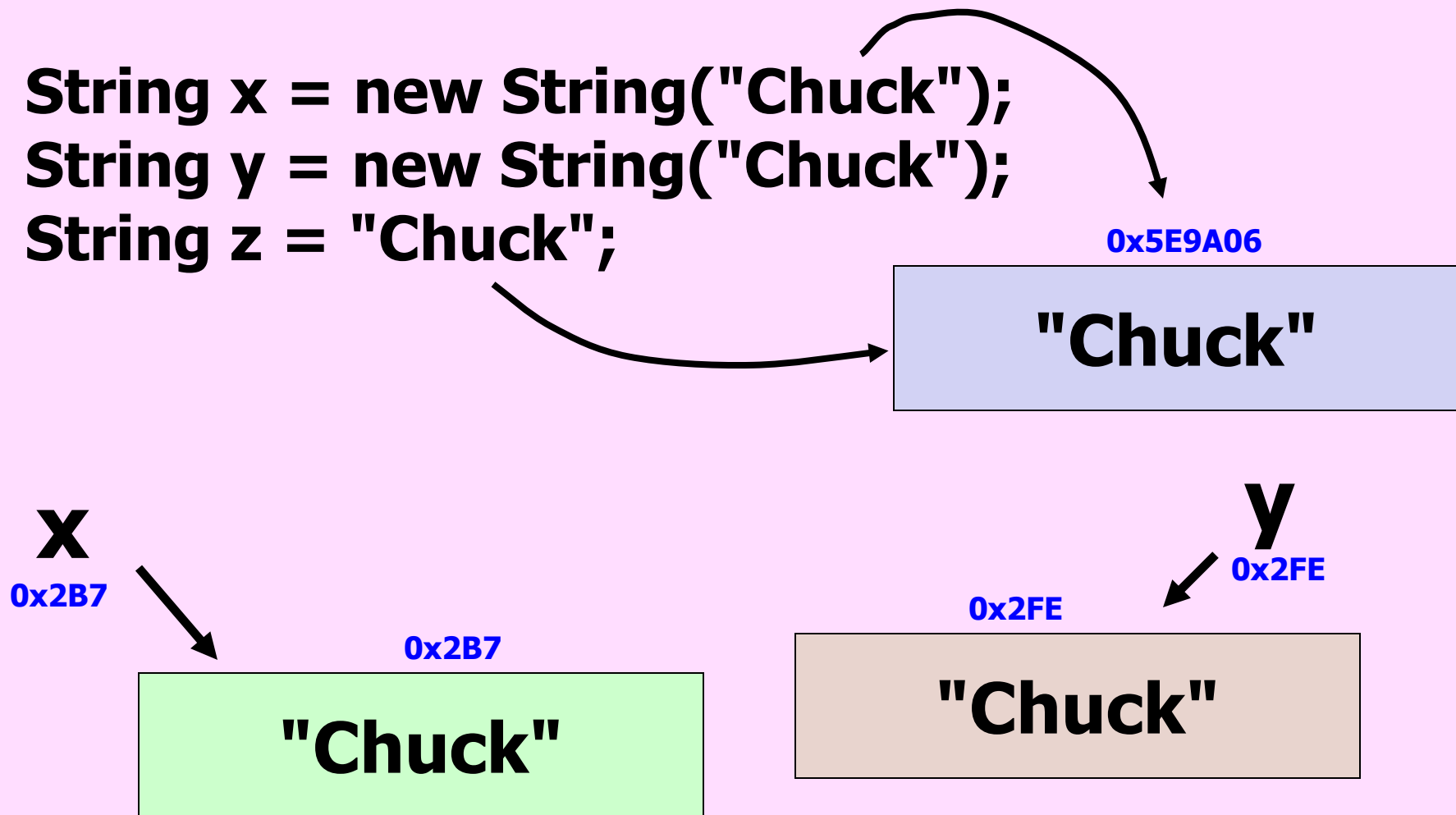
Contest Puzzlers

```
String x = new String("Chuck");  
String y = new String("Chuck");
```



Contest Puzzlers

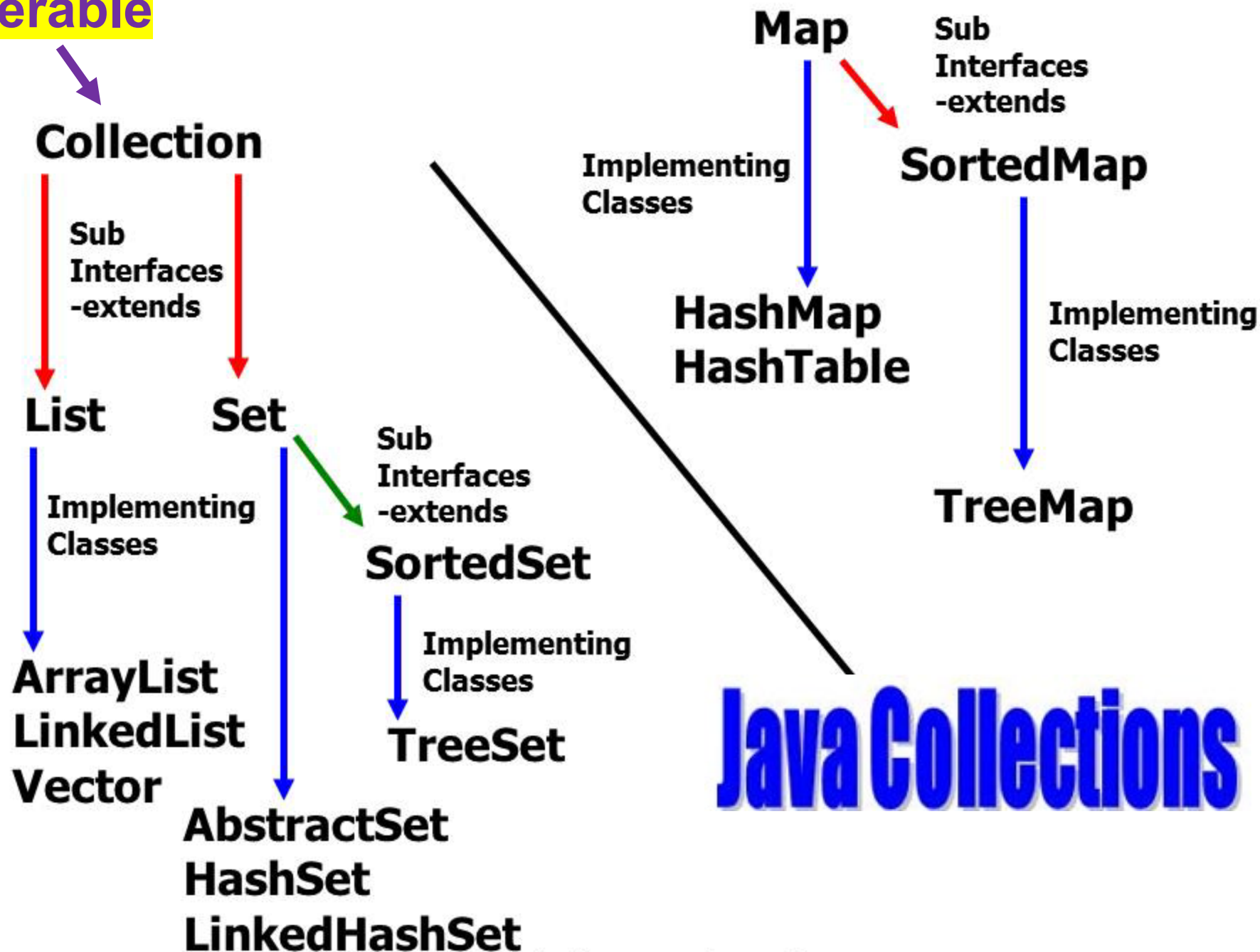
```
String x = new String("Chuck");  
String y = new String("Chuck");  
String z = "Chuck";
```



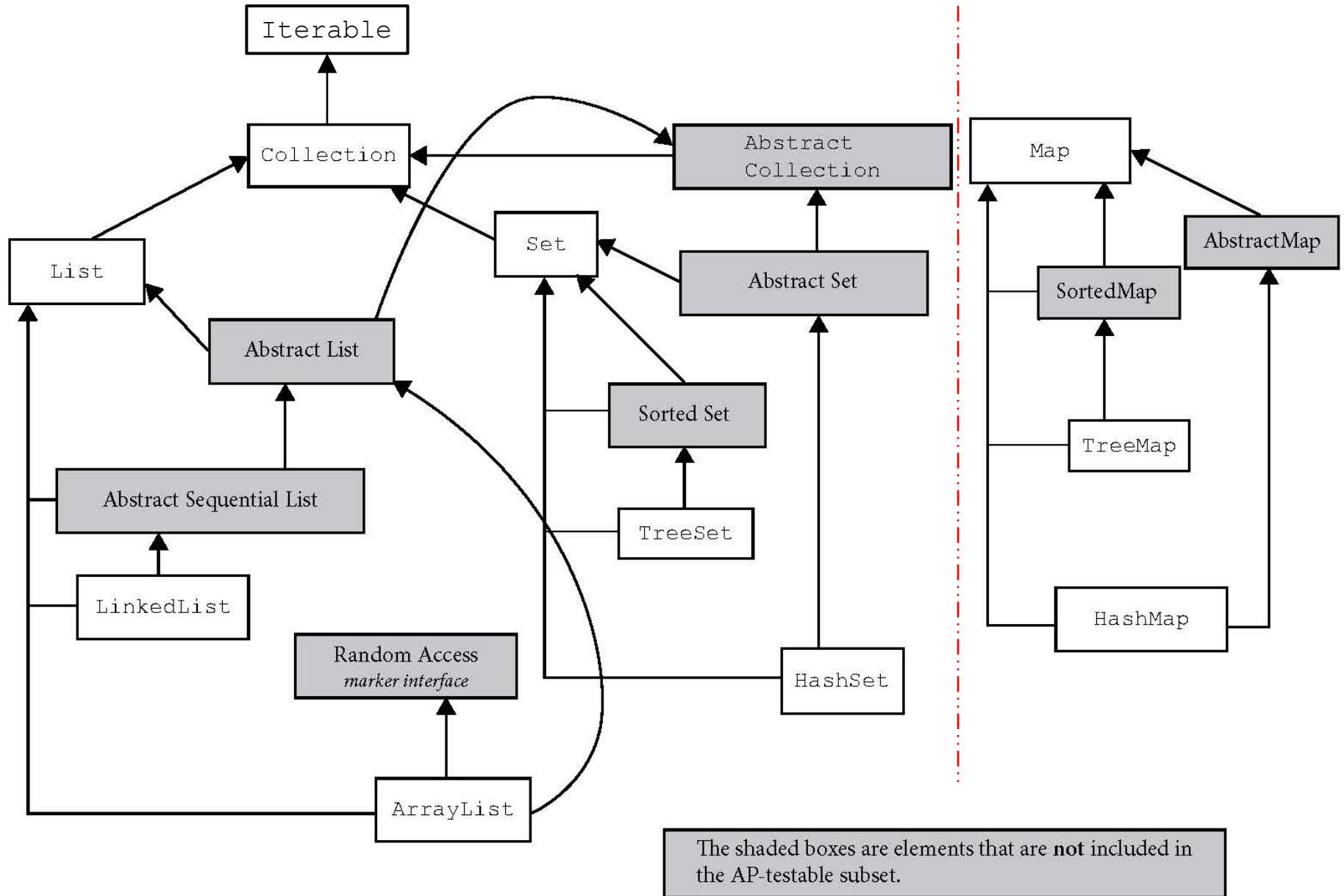
references.java

Iterators

Iterable

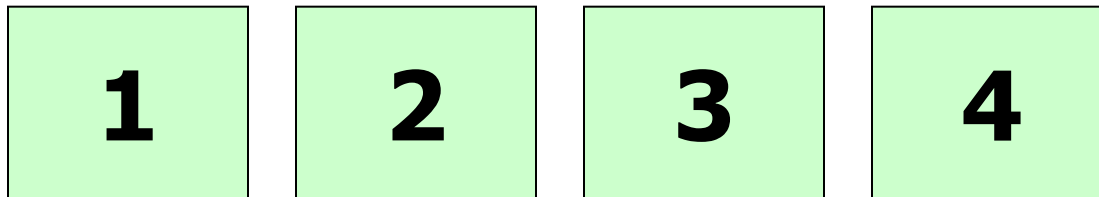


The Java Collections Hierarchy



What is an Iterator?

An Iterator provides a standard way to access all the references stored in a collection.

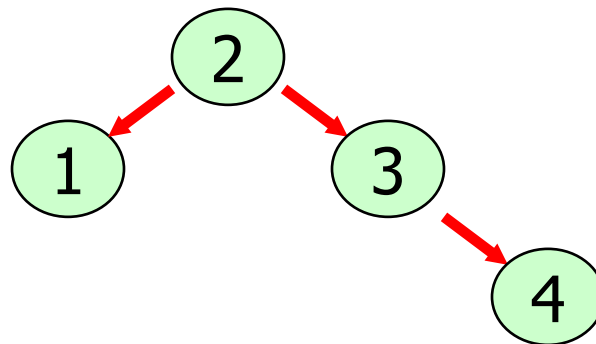




What is an Iterator?

Index-based loops can only be used with Lists, since Collections such as TreeSets, HashSets, and Priority Queues are not stored sequentially.

Iterators, on the other hand, can be used with any kind of Collection (Lists, Queues, and Sets).



Java Iterators

You don't explicitly instantiate an iterator, you ask for one.

Every Collection has a method that returns an iterator, which means every List, Queue, and Set have an `iterator()` method:

```
Iterator iter1 = list.iterator();  
Iterator iter2 = set.iterator();  
Iterator iter3 = queue.iterator();
```

Java Iterators

Map does not have an iterator, but it does have methods that return Collections whose iterators you can use.

Iterator

Interface



next

You don't call a constructor to create an iterator. You create your collection and then ask it for an iterator:

```
ArrayList<String> words;  
words = new ArrayList<String>();  
words.add("at");  
words.add("is");  
words.add("of");
```

```
Iterator<String> it = words.iterator();
```

Iterator

frequently used methods

Name	Use
E next()	returns a reference to the next item
void remove()	removes the last ref returned by next
boolean hasNext()	checks to see there are more items

```
import java.util.Iterator;
```



next()

```
ArrayList<String> words;  
words = new ArrayList<String>();  
words.add("at");  
words.add("is");  
words.add("of");  
words.add("us");
```

OUTPUT
at

```
Iterator<String> it = words.iterator();  
System.out.println(it.next());
```



next()

```
ArrayList<String> words;  
words = new ArrayList<String>();  
words.add("at");  
words.add("is");  
words.add("of");  
words.add("us");
```

OUTPUT

**at
is
of
us**

```
Iterator<String> it = words.iterator();  
System.out.println(it.next());  
System.out.println(it.next());  
System.out.println(it.next());  
System.out.println(it.next());
```



next()

```
ArrayList<String> words;  
words = new ArrayList<String>();  
words.add("at");
```

```
Iterator<String> it = words.iterator();  
System.out.println(it.next());  
System.out.println(it.next());
```

OUTPUT

at

error

A NoSuchElementException is thrown.

iteratorone.java

hasnext.java

hasNext()

```
ArrayList<String> words;  
words = new ArrayList<String>();
```

```
words.add("at");  
words.add("is");  
words.add("of");
```

OUTPUT

at
is
of

```
Iterator<String> it = words.iterator();  
while(it.hasNext())  
{  
    System.out.println(it.next());  
}
```


hasNext()

Note that if you don't use generics, Java only knows it's some kind of Object.

```
ArrayList<String> words;  
words = new ArrayList<String>();
```

```
words.add("at");  
words.add("is");  
words.add("of");
```

This would
output the
same thing:

```
Iterator it = words.iterator();  
while(it.hasNext())  
{  
    System.out.println(it.next());  
}
```

OUTPUT

at
is
of

hasNext()

But I couldn't call any String methods:

```
Iterator it = words.iterator();  
while(it.hasNext())  
{  
    System.out.println(it.next().length());  
}
```

hasNext()

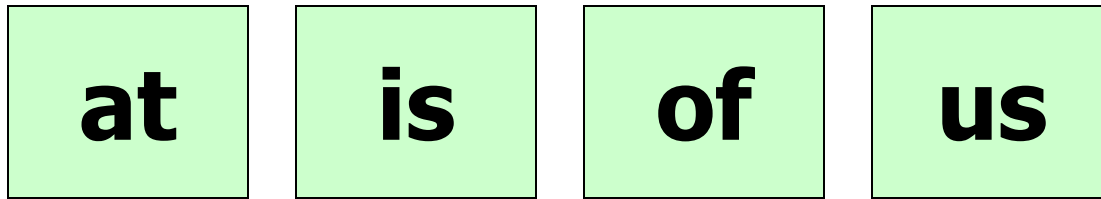
Now I can:

```
Iterator<String> it = words.iterator();  
while(it.hasNext())  
{  
    System.out.println(it.next().length());  
}
```

Inside
the
next()
method

next()

list



```
Iterator it = list.iterator();
```

```
curr = null  
next = "at"
```



next()

method next()

{

curr = next

next = *next ref in the collection*

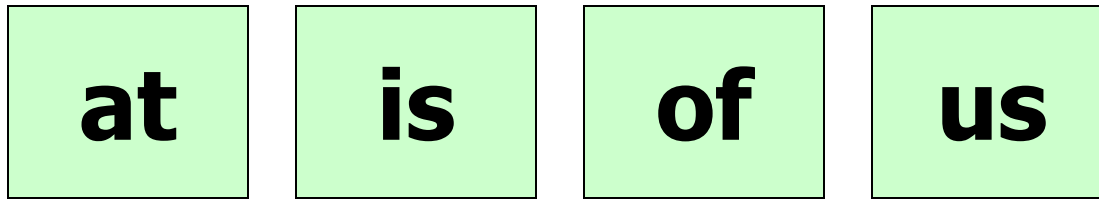
return curr

}

next()

curr = "at"
next = "is"

list



it

```
method next()  
{
```

```
    curr = next
```

```
    next = next ref in the collection
```

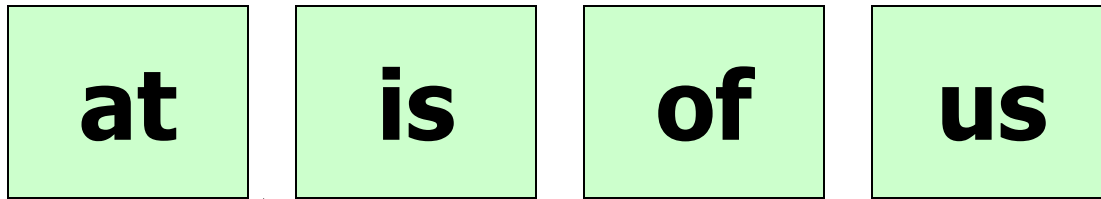
```
    return curr
```

```
}
```



next()

list



it

it.next();

**next moves the iterator
up one spot and returns a
reference to the 1st item.**

removed

The Iterator remove method removes the last element iterated over.

I.e. the last element returned by the next() method.

removed

```
ArrayList<String> words;  
words = new ArrayList<String>();
```

```
words.add("at");  
words.add("is");  
words.add("of");
```

OUTPUT

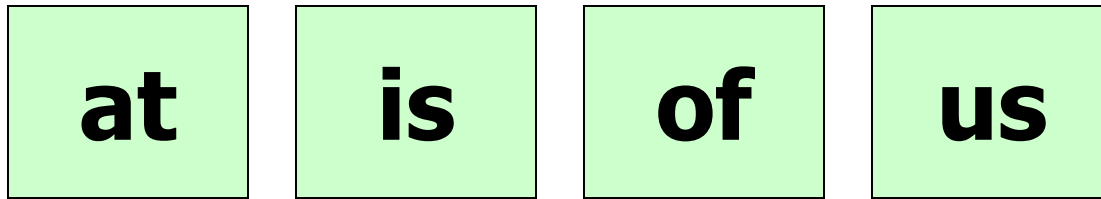
at
is
[is, of]

```
Iterator<String> it = words.iterator();  
System.out.println(it.next());  
it.remove();  
System.out.println(it.next());  
System.out.println(words);
```



removed

list



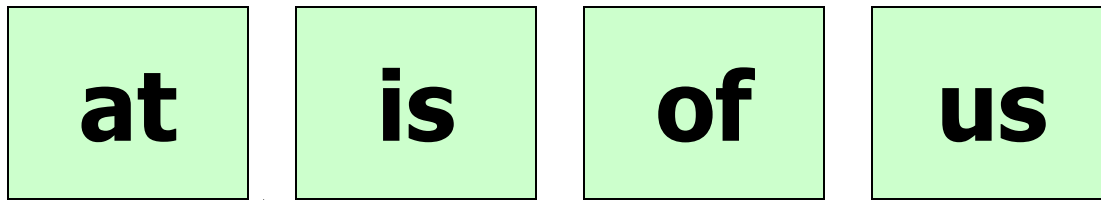
it

```
Iterator it = list.iterator();
```



removed

list

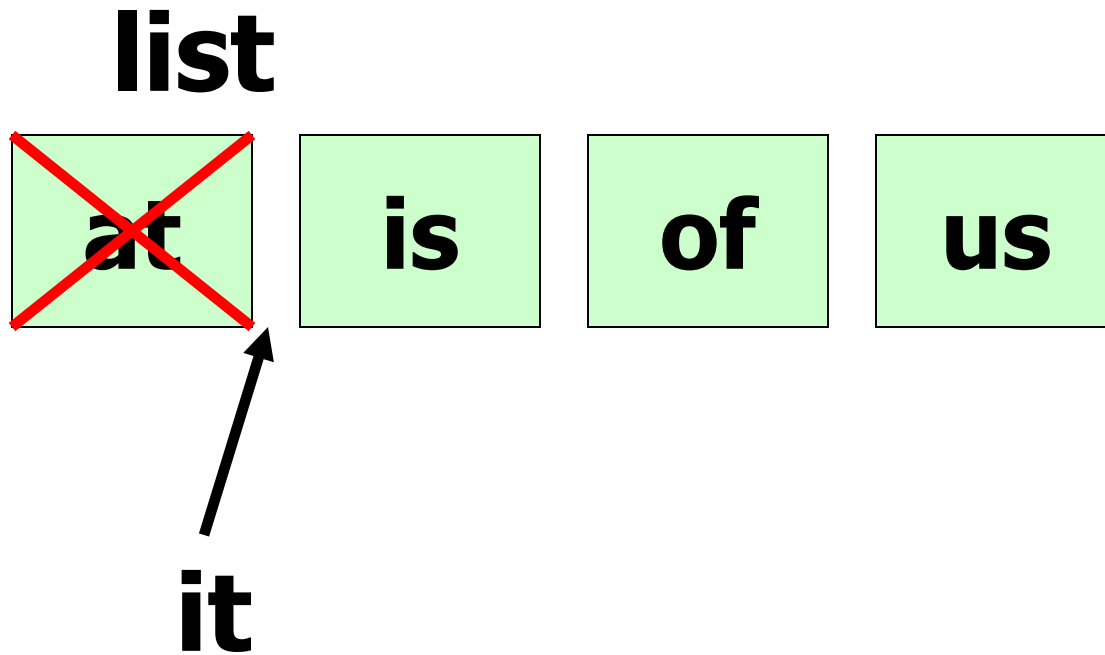


it

it.next();

**next moves the iterator
one spot and returns a
reference to at.**

removed



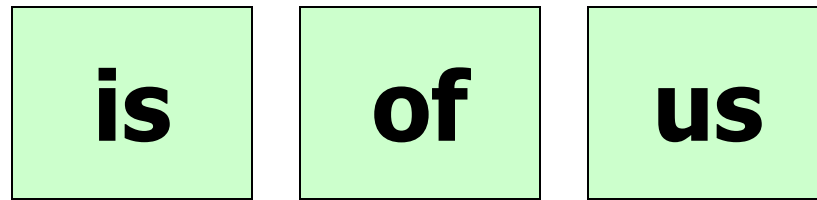
it.remove();

**remove always modifies
the last reference
returned by next.**



removed

list



it.next();

**next moves the iterator
one spot and returns a
reference to **is**.**

removed

```
ArrayList<String> words;  
words = new ArrayList<String>();
```

```
words.add("at");  
words.add("is");  
words.add("of");
```

OUTPUT

at

error

```
Iterator<String> it = words.iterator();  
out.println(it.next());  
it.remove();  
it.remove();
```

curr = null, next = "at"
curr = "at", next = "is"
curr = null, next = "is"

removed

list

at

is

of

us

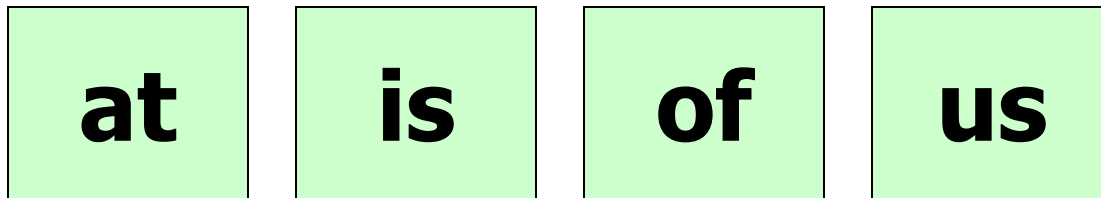
it





removed

list



it

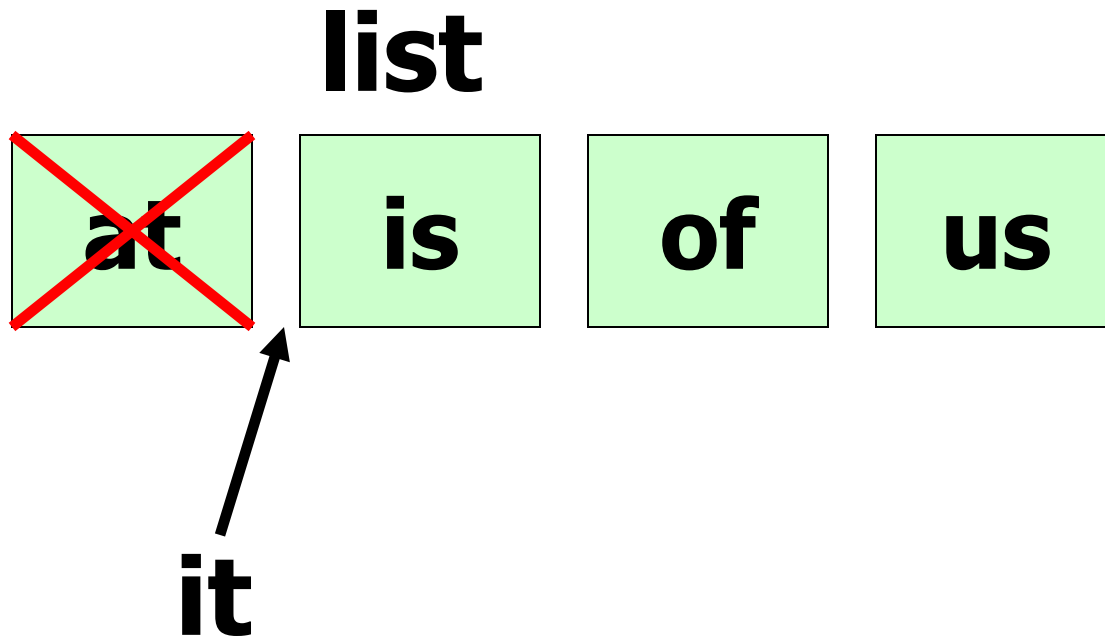


A black arrow points from the word 'it' to the 'is' box in the list above.

it.next();

**next moves the iterator
up one spot and returns a
reference to the 1st item.**

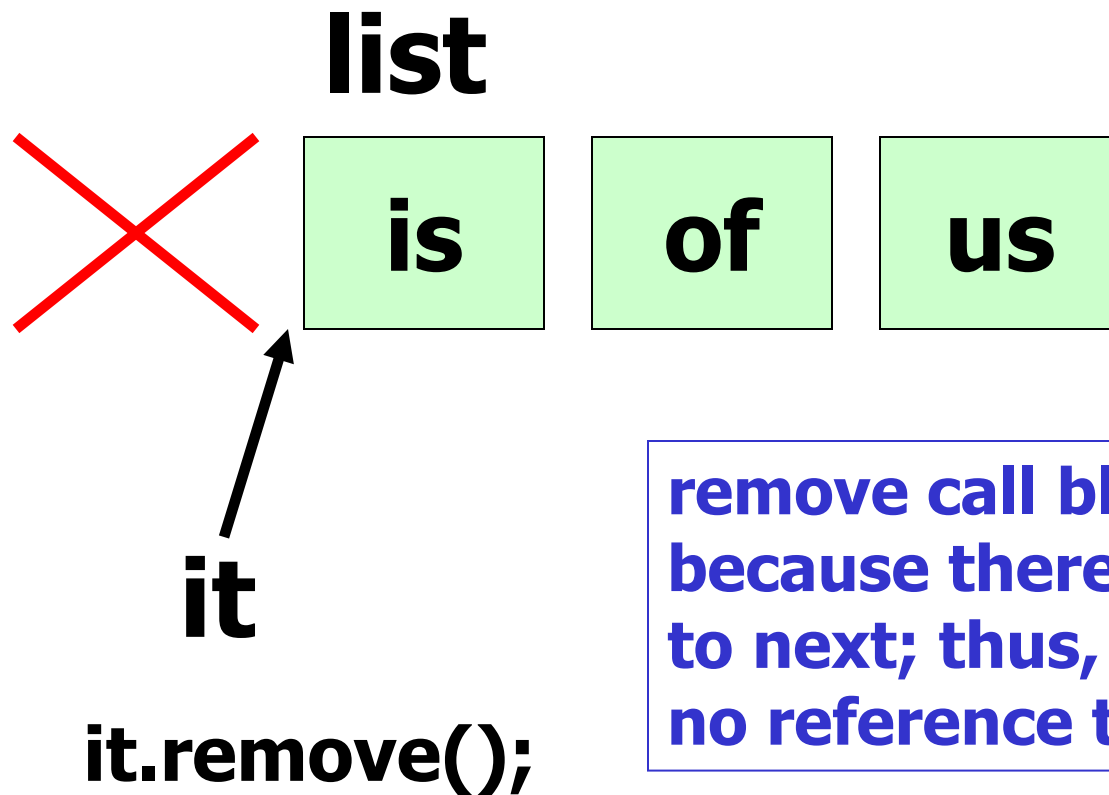
removed



it.remove();

**remove always modifies
the last reference returned
by next.**

removed



remove call blows up
because there was no call
to next; thus, there was
no reference to modify.

Throws an IllegalStateException

AN ITERATOR DOESN'T STORE ANYTHING!

**When you make an iterator,
it's not storing a separate
copy of the list. It simply
manages the list for you
and expects you to use it
exclusively.**



ConcurrentModificationException

```
ArrayList<String> words;  
words = new ArrayList<String>();
```

```
words.add("at");  
words.add("is");  
words.add("of");
```

```
Iterator<String> it = words.iterator();  
System.out.println(it.next());  
words.remove(1);  
System.out.println(it.next());
```

OUTPUT

at

error



No error if you don't use Iter

```
ArrayList<String> words;  
words = new ArrayList<String>();
```

```
words.add("at");  
words.add("is");  
words.add("of");
```

OUTPUT

at

```
Iterator<String> it = words.iterator();  
System.out.println(it.next());  
words.remove(1);
```



Okay ... but not okay

```
ArrayList<String> words;  
words = new ArrayList<String>();
```

```
words.add("at");  
words.add("is");  
words.add("of");
```

```
Iterator<String> it = words.iterator();  
System.out.println(it.next());  
words.remove(1);  
it = words.iterator();  
System.out.println(it.next());  
System.out.println(it.next());
```

OUTPUT

at
at
of

What wrong with this?

```
List <String>list = new ArrayList<String>();  
list.add("1");  
list.add("2");  
list.add("3");  
Iterator <String>iter = list.iterator ();  
iter.remove();  
iter = list.iterator();  
while (iter.hasNext())  
    System.out.print((String)iter.next());
```


IllegalStateException is thrown

You must call next() before remove()!

```
List <String>list = new ArrayList<String>();  
list.add("1");  
list.add("2");  
list.add("3");  
Iterator <String>iter = list.iterator ();  
iter.remove();  
iter = list.iterator();  
while (iter.hasNext())  
    System.out.print((String)iter.next());
```



What's wrong with this loop?

```
int totalLengths(ArrayList<String> list) {  
    Iterator<String> iter = data.iterator();  
    int total = 0;  
    while (iter.hasNext()) {  
        if (iter.next().length() < 5)  
            total += iter.next().length();  
    }  
    return total;  
}
```

The fix: call next() once

```
int totalLengths(ArrayList<String> list) {  
    Iterator<String> iter = data.iterator();  
    int total = 0;  
    while (iter.hasNext()) {  
        String word = iter.next();  
        if (word.length() < 5)  
            total += word.length();  
    }  
    return total;  
}
```

removeone.java

removetwo.java

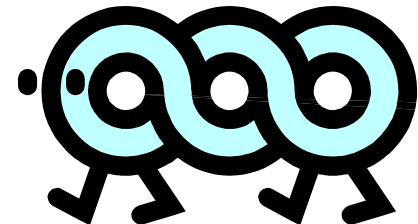
the new for loop

new for loop

In an enhanced for loop, Java will automatically create an Iterator and use it to iterate through the array or Collection.

for (int num : array)

for (Integer value : list)



new for loop

```
ArrayList<Integer> list;  
list = new ArrayList<Integer>();  
list.add(3);  
list.add(9);
```

```
for (Integer num : list)  
    System.out.print(num + " ");
```

OUTPUT

3 9 2

newforone.java

Start work on Lab 5